Secure-Document-Vault

This project is about a Secure Document Vault. The user accesses the web application via a website, log in or registers. After that they can upload or download files which will be encrypted and stored in the database.

Technologies

Frontend

- Simple client-side rendering, no fancy animations, simple, but usable bootstrap site
- HTML, CSS, JS, BootStrap

Backend

- Node.JS Runtime with TypeScript
- Express REST API for web framework

Third-Party

- Twilio for email, sms notification

Database

- MySQL 8
  All ends will run in docker containers

Cryptography

- TLS 1.3
- AES-256-GCM for file encryption
- npm Argon2id library for password storage
- JWT with HMAC-SHA256 for authenticated sessions (npm jsonwebtoken)
- npm crypto for general hashing and random key/IV generation
- *Post-Quantum:* ML-KEM for hybrid key encapsulation in future

Hosting

- Containerized with Docker Compose
- Frontend, Backend, DB separated
- Hosted possibly on DigitalOcean Droplet or self host

Domain

- vault.local for local hosting

- possibly more production ready domain in case of cloud hosting

Certificates

- Self-signed with OpenSSL

Functionalities

- Login, Authentication
- File upload
- File download

Data flows

Login/Auth

Client → POST /api/login → Backend asks DB, validates → returns JWT token

File upload

Client → POST /api/upload (file + JWT) → Backend encrypts with AES-GCM → ciphertext + metadata stored in DB → returns confirmation/error message

File download

Client → GET /api/download/@{id} + JWT → Backend verifies with JWT → decrypts file → sends back decrypted file

SMS

Backend → Twilio API POST → Send sms to user after upload/download